

# ElectroSim-CV Analysis Suite

A publication-ready toolkit for cyclic voltammetry (CV) recorded at multiple scan rates — seven complementary analyses sharing one uploaded file.

Maintained by **Rajeev Kumar** ([rkumar@nccu.edu](mailto:rkumar@nccu.edu)), North Carolina Central University.

Single canonical README for the whole project — end-user, web-demo, sample-data, build, and installer docs are all here.

## What it does

Seven scan-rate CV analyses, each in its own tab, all reading the **same uploaded file**:

| Analysis                          | What it extracts                                                                        | Regime                    |
|-----------------------------------|-----------------------------------------------------------------------------------------|---------------------------|
| <b>Dunn's Method</b>              | Power-law $b$ -value charge-storage classification + capacitive/diffusive current split | any                       |
| <b>Trasatti Analysis</b>          | Inner / outer / total voltammetric charge (site accessibility)                          | pseudocapacitive / porous |
| <b>ECASA</b>                      | Double-layer capacitance $C_{dl}$ → electrochemically active surface area               | non-Faradaic              |
| <b>Areal Capacitance</b>          | Areal capacitance $C_A$ ( $F\ cm^{-2}$ ) + rate-capability / retention                  | capacitive                |
| <b>Randles-Sevcik</b>             | Diffusion coefficient $D$ from peak current vs $\sqrt{v}$ (scan rate)                   | Faradaic                  |
| <b>Peak Area / Charge</b>         | Charge $Q$ , surface coverage $\Gamma$ , moles $N$ , $Q_a/Q_c$ ratio                    | Faradaic                  |
| <b>Electron-Transfer Kinetics</b> | Heterogeneous rate constant $k^0$ (Nicholson) / $\alpha$ , $k_s$ (Laviron)              | Faradaic                  |

Tab order: **Dunn** → **Trasatti** → **ECASA** → **Areal** → **Randles-Sevcik** → **Peak Area** → **Electron-Transfer Kinetics**, each grouped beside its closest companion. Faradaic tabs gate on genuine redox-peak detection and refuse to run on non-Faradaic data.

Available in **three forms**, all with the same analyses:

1. **Hosted web demo (Streamlit)** — fastest way in, nothing to install: [electrosim-cvanalysis-suite.streamlit.app](https://electrosim-cvanalysis-suite.streamlit.app)
2. **Web app (PyWebView desktop window)** — the same Streamlit app in a native window, fully offline.
3. **Tkinter desktop GUI** — lightweight native fallback.

## Getting started

**Web demo:** open <https://electrosim-cvanalysis-suite.streamlit.app/>, upload a CV file (wide Excel or stacked CSV), get all seven analyses instantly.

**Desktop install (Windows 64-bit):** download [ElectroSim-CVAnalysis-Suite-Setup-2.1.exe](#) from the [Releases](#) page → double-click → 3 screens (Welcome → Installing → Finish, ~15 s, no admin rights). Inside the app, click **Load files...** → pick a template from `sample_data/` → switch to the analysis tab you need.

New / non-technical? See [QUICK\\_START.md](#) for a 1-page plain-English walkthrough (including the SmartScreen prompt). Then [docs/user\\_guide.md](#) for the full walkthrough and [docs/methods.md](#) for the scientific background.

## Verifying the download

`Setup-2.1.exe` is unsigned, so Windows SmartScreen shows "Windows protected your PC" the first time — click **More info** → **Run anyway**. To confirm the file is genuine, check its SHA-256:

```
Get-FileHash -Algorithm SHA256 $env:USERPROFILE\Downloads\ElectroSim-CVAnalysis-Suite-Setup-2.1.exe
```

Expected (case-insensitive): `10A0DB7F8BCAC53C92206BBA28C99CA89FF6B4D9761043EB049F5265A6FC1294`

## System requirements

- Windows 10 1809 (build 17763) or later, 64-bit
- ~700 MB free disk space
- Edge WebView2 (preinstalled on Windows 10/11; required only for the hybrid build — the Tkinter fallback works without it)

The source code ( `.py` ) is intentionally not published; the suite is distributed in binary form only. See [LICENSE](#) and [NOTICE](#).

## Input data formats

| Format                     | Layout                                                                                                                         |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Wide Excel</b>          | Header-encoded scan rate, one column pair per rate (e.g. <code>Potential at 50 mV/s</code> , <code>Current at 50 mV/s</code> ) |
| <b>Stacked CSV / Excel</b> | Three columns: Potential, Current, ScanRate                                                                                    |
| <b>Single CV</b>           | Two columns: Potential, Current (Dunn / ECASA / RS need multiple scan rates)                                                   |

## Sample templates

Ready-to-load templates live in [release/sample\\_data/](#) — open any from the **Data Files** tab:

| File                      | Purpose                                                 |
|---------------------------|---------------------------------------------------------|
| Dunn method template.xlsx | Dunn's Method, wide Excel with annotated two-row header |
| ECASA CV template.xlsx    | ECASA non-Faradaic window, wide Excel two-row header    |

**Using a template?** Paste your data in and upload as-is. **Do not modify the first two rows** — row 1 (scan-rate headers) and row 2 (Potential / Current sub-headers) are required. Add or remove data rows freely.

**Recommended workflow:** open `Dunn method template.xlsx` in **Data Files** → **Data Plotting** → **Plot** (confirm the CV looks right) → **Dunn's Method** → set branch and prominence → **Run analysis**.

## Running the web app locally

The web app lives in [release/web-demo/](#). It is a Streamlit app and **must** be launched with `streamlit run`, not plain `python`:

```
cd release\web-demo

# RIGHT — Streamlit boots its runtime, then exec's the loader
streamlit run electrosim_app.py

# RIGHT — pick a specific Python version (must match a shipped .pyc; see below)
py -3.12 -m streamlit run electrosim_app.py    # cp312 engine
py -3.14 -m streamlit run electrosim_app.py    # cp314 engine

# WRONG — plain python prints a friendly error and exits
python electrosim_app.py
```

**Why `streamlit run`?** The engine calls `st.set_page_config`, `st.sidebar.*`, etc. at module-load time, which need Streamlit's per-thread `ScriptRunContext`. The loader detects a missing context and prints a clear "WRONG / RIGHT" message instead of an internals traceback.

**Pick the right Python** — one that has **both** Streamlit **and** a matching engine `.pyc` (currently `cp312` for 3.12, `cp314` for 3.14). If deps are missing:

```
py -3.14 -m pip install -r release\web-demo\requirements.txt
```

 A running Streamlit server loads the engine bytecode **once at startup** and does not hot-reload a recompiled `.pyc` — **restart the server** after any engine change.

## How the web app is packaged (bytecode-only)

The Python in `release/web-demo/` is **compiled bytecode**:

- `electrosim_app.py` — small Streamlit entry-point loader (the only `.py`, no analysis logic). This is what Streamlit Cloud runs.
- `electrosim_engine.cp312.pyc` / `electrosim_engine.cp314.pyc` — the analysis engine, one per supported Python version. The loader auto-picks the match for the running interpreter and `exec()` s it.

**Python-version pinning.** A `.pyc` is keyed to the exact interpreter version. If Streamlit Cloud upgrades and the demo fails with a "magic number" error, recompile the upstream source ( `py -3.XX -m py_compile <upstream-source>.py` ) and drop `__pycache__/*.cpython-3XX.pyc` in as `electrosim_engine.cp3XX.pyc` .

---

## Citation

If you use this in academic work, please cite via the metadata in [CITATION.cff](#). A DOI is issued for each release through [Zenodo](#).

---

## License & permitted use

Copyright (c) 2026 Rajeev Kumar. All rights reserved. Binary / compiled-bytecode distribution only, attribution required. See [LICENSE](#) and [NOTICE](#).

- **Permitted:** personal use (desktop or hosted demo) for evaluation and learning; academic / non-commercial research, including publications, provided the Suite is cited.
- **Not permitted without prior written permission:** commercial use of the app, demo, or their outputs; re-deployment under any other name or hosting; decompilation, disassembly, or reverse-engineering of the bytecode.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND. Contact: [rkumar@nccu.edu](mailto:rkumar@nccu.edu).

---

## For maintainers — building from source

End users don't need this — they download the prebuilt installer. The analysis source is **not** in this public repo; the maintainer builds from a private tree. Building in a clone of the public repo fails with "module not found" — expected.

**Build reference files** (tracked in [release/build/](#)):

| File                                     | What it is                                                                                                                                                         |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ElectroSim-CVAnalysis-Suite.spec         | PyInstaller config for the <b>PyWebView/Streamlit hybrid</b> build (main app)                                                                                      |
| ElectroSim-CVAnalysis-Suite-Tkinter.spec | PyInstaller config for the <b>Tkinter</b> fallback build                                                                                                           |
| electrosim_build.ps1                     | PowerShell wrapper running PyInstaller against both specs ( <code>-Target streamlit\ tkinter\ all</code> , optional <code>-Clean</code> , <code>-SyncDeps</code> ) |

The hybrid spec freezes the Streamlit launcher and the seven-analysis app; the Tkinter spec freezes the native Tkinter GUI and its analysis module. The hosted-demo engine ( `release/web-demo/electrosim_engine.cp3XX.pyc` ) is compiled separately — see *How the web app is packaged* above.

**Full release workflow** (from the private source tree at the project root):

```
# 1. Build both PyInstaller bundles -> dist\ at the project root
#   (-SyncDeps first installs scipy/plotly/kaleido/openpyxl/pywebview/... from requirements.txt)
#   .\release\build\electrosim_build.ps1 -Clean -SyncDeps

# 2. Install Inno Setup 6 (once): winget install --id JRSoftware.InnoSetup
# 3. Compile the installer
& "${env:LocalAppData}\Programs\Inno Setup 6\ISCC.exe" release\installer\electrosim_setup.iss
# -> release\installer\Output\ElectroSim-CVAnalysis-Suite-Setup-X.Y.exe (upload to GitHub Releases)
```

**What the installer produces:** a 3-screen, no-UAC, per-user install into

`%LocalAppData%\Programs\ElectroSim-CV Analysis Suite\` with a Desktop shortcut (hybrid build), a Start Menu folder (main app, User Guide, Sample Data, Tkinter fallback + Uninstall under `Tools\`), and `LICENSE / NOTICE / README.md / CHANGELOG.md / CITATION.cff / sample_data/ / docs/` at the install root. The Tkinter build is a fallback for when the hybrid can't launch (e.g. broken Edge WebView2).

**Cutting a new release (2.1, 3.0, ...):**

1. Bump `MyAppVersion` in `release/installer/electrosim_setup.iss`.
2. Update [CHANGELOG.md](#) (shown on the post-install screen).
3. **Do NOT change** `AppId` — its GUID is what makes Windows upgrade in place instead of side-by-side.
4. Rebuild bundles, recompile the `.iss`, upload the new `Setup-X.Y.exe`.
5. **Recompute the SHA-256** of the new `.exe` and update it in this README and in [QUICK\\_START.md](#).
6. **Recommended:** submit the new unsigned `Setup.exe` to [Microsoft](#) so SmartScreen/Defender clears it (24–72 h; re-submit per release since the hash changes).

**Build troubleshooting:** ISCC error: Source file not found: `..\..\dist\...` → run PyInstaller first / run ISCC from the project root. ISCC: command not found → Inno Setup not installed or not on PATH. Installer crashes on a clean VM → incomplete PyInstaller bundle (run the `.exe` from `dist\...\` directly to confirm). Two Add/Remove Programs entries after upgrade → someone changed `AppId`.

---

## Support / bug reports

---

Open an [Issue](#). Please include: the exact build name from **Help** → **About**, the input file format (anonymise data if needed), and a screenshot of the error if applicable.